NAME- INSIYA ZAIDI

ROLL NO – 23BEC073

B.TECH – ECE

SUBJECT - DBMS

SUBMITTED TO – DR. WASEEM

INDEX -

S.NO	TOPIC	DATE
1	Database Creation , Table Creation / Insertion	1/8/25
2	Primary Key Constraints	8/8/25
3	Foreign Key Constraints	8/8/25
4	Alter Table, Drop Table, Update Table	22/8/25
5	Aggregate Function	22/8/25
6	Join	10/10/25
7	Subquery	17/10/25
8	Nested Query	24/10/25

PRACTICAL NO -1

TOPIC – DATABASE CREATION , TABLE CREATION /INSERTION

1)

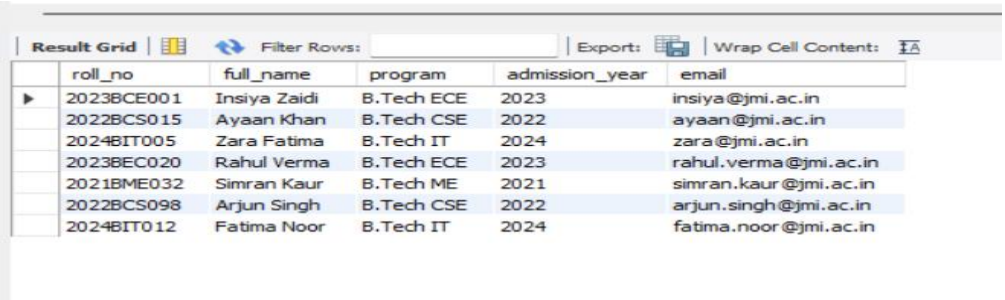
```
CREATE DATABASE student_db;

USE student_db;

CREATE TABLE students (
    roll_no VARCHAR(12) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    program VARCHAR(60),
    admission_year SMALLINT,
    email VARCHAR(120)
);

INSERT INTO students (roll_no, full_name,   program, admission_year, email) VALUES

('2023BCE001', 'Insiya Zaidi', 'B.Tech ECE',   2023,   'insiya@jmi.ac.in'),

('2022BCS015', 'Ayaan Khan',  'B.Tech CSE',   2022, 'ayaan@jmi.ac.in'),

('2024BIT005', 'Zara Fatima', 'B.Tech IT',  2024, 'zara@jmi.ac.in'),

('2023BEC020', 'Rahul Verma', 'B.Tech ECE', 2023,
'rahul.verma@jmi.ac.in'),
```

('2021BME032', 'Simran Kaur', 'B.Tech ME', 2021, 'simran.kaur@jmi.ac.in'),

('2022BCS098', 'Arjun Singh', 'B.Tech CSE', 2022, 'arjun.singh@jmi.ac.in'),

('2024BIT012', 'Fatima Noor', 'B.Tech IT', 2024, 'fatima.noor@jmi.ac.in');

SELECT * FROM students;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

roll_no	full_name	program	admission_year	email
2023BCE001	Insiya Zaidi	B.Tech ECE	2023	insiya@jmi.ac.in
2022BCS015	Ayaan Khan	B.Tech CSE	2022	ayaan@jmi.ac.in
2024BIT005	Zara Fatima	B.Tech IT	2024	zara@jmi.ac.in
2023BEC020	Rahul Verma	B.Tech ECE	2023	rahul.verma@jmi.ac.in
2021BME032	Simran Kaur	B.Tech ME	2021	simran.kaur@jmi.ac.in
2022BCS098	Arjun Singh	B.Tech CSE	2022	arjun.singh@jmi.ac.in
2024BIT012	Fatima Noor	B.Tech IT	2024	fatima.noor@jmi.ac.in

2)

CREATE DATABASE company_db;

USE company_db;

CREATE TABLE departments (

dept_code VARCHAR(10) NOT NULL,

dept_name VARCHAR(100),

location VARCHAR(50)

);

INSERT INTO departments (dept_code, dept_name, location) VALUES

('D001', 'Human Resources', 'Delhi'),('D002', 'Finance', 'Mumbai'),('D003', 'Engineering', 'Bangalore'),('D004', 'Marketing', 'Kolkata'),('D005', 'IT Support', 'Hyderabad');

CREATE TABLE employees (

emp_id VARCHAR(10) NOT NULL,emp_name VARCHAR(100), job_title VARCHAR(60),dept_code VARCHAR(10),salary INT);

INSERT INTO employees (emp_id, emp_name, job_title, dept_code, salary) VALUES

('E101', 'Rahul Sharma', 'HR Executive', 'D001', 35000),

('E102', 'Priya Gupta', 'Accountant', 'D002', 42000),

('E103', 'Aman Verma', 'Software Engineer', 'D003', 62000),

('E104', 'Sneha Patel', 'Marketing Specialist', 'D004', 39000),

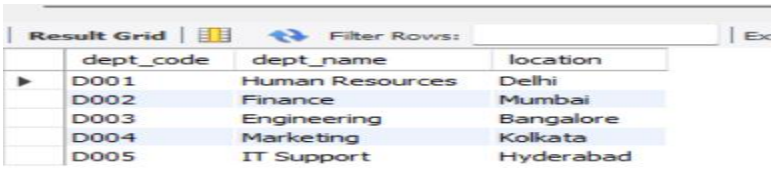
('E105', 'Zaid Khan', 'Network Technician', 'D005', 45000),

('E106', 'Ananya Rao', 'Data Analyst', 'D003', 58000),

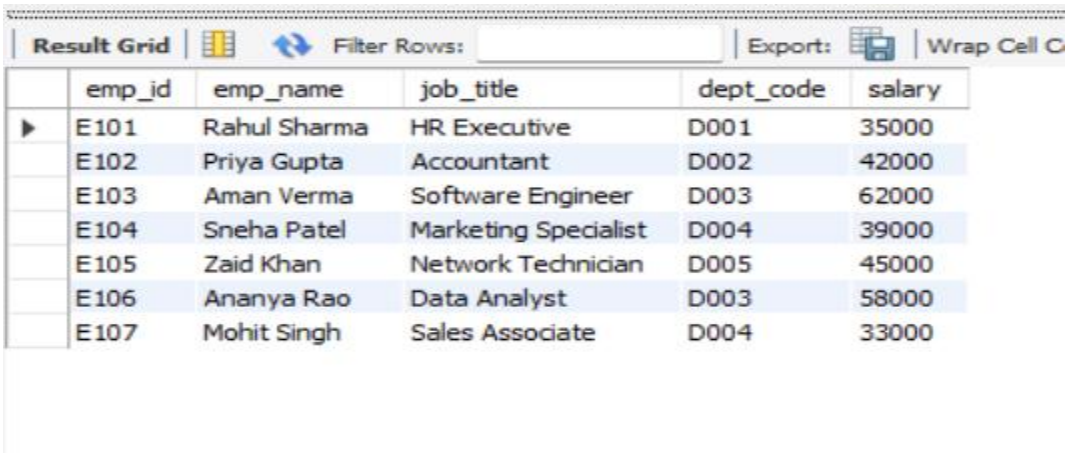
('E107', 'Mohit Singh', 'Sales Associate', 'D004', 33000);

SELECT * FROM departments;

SELECT * FROM employees;



dept_code	dept_name	location
D001	Human Resources	Delhi
D002	Finance	Mumbai
D003	Engineering	Bangalore
D004	Marketing	Kolkata
D005	IT Support	Hyderabad



Result Grid | Filter Rows: | Export: | Wrap Cell C

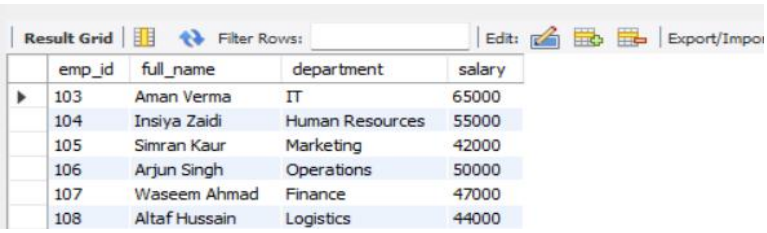
emp_id	emp_name	job_title	dept_code	salary
E101	Rahul Sharma	HR Executive	D001	35000
E102	Priya Gupta	Accountant	D002	42000
E103	Aman Verma	Software Engineer	D003	62000
E104	Sneha Patel	Marketing Specialist	D004	39000
E105	Zaid Khan	Network Technician	D005	45000
E106	Ananya Rao	Data Analyst	D003	58000
E107	Mohit Singh	Sales Associate	D004	33000

PRACTICAL NO -2

TOPIC –PRIMARY KEY CONSTRAINT

1)

```
CREATE DATABASE employee_db;

USE employee_db;

CREATE TABLE employees (

 emp_id INT PRIMARY KEY,full_name VARCHAR(100) NOT NULL,
department VARCHAR(60),salary INT);

INSERT INTO employees (emp_id, full_name, department, salary)
VALUES

(103, 'Aman Verma', 'IT', 65000),(104, 'Insiya Zaidi', 'Human Resources',
55000),  (105, 'Simran Kaur', 'Marketing', 42000),(106, 'Arjun Singh',
'Operations', 50000),(107, 'Waseem Ahmad', 'Finance', 47000),(108,
'Altaf Hussain', 'Logistics', 44000);

SELECT * FROM employees;
```



emp_id	full_name	department	salary
103	Aman Verma	IT	65000
104	Insiya Zaidi	Human Resources	55000
105	Simran Kaur	Marketing	42000
106	Arjun Singh	Operations	50000
107	Waseem Ahmad	Finance	47000
108	Altaf Hussain	Logistics	44000

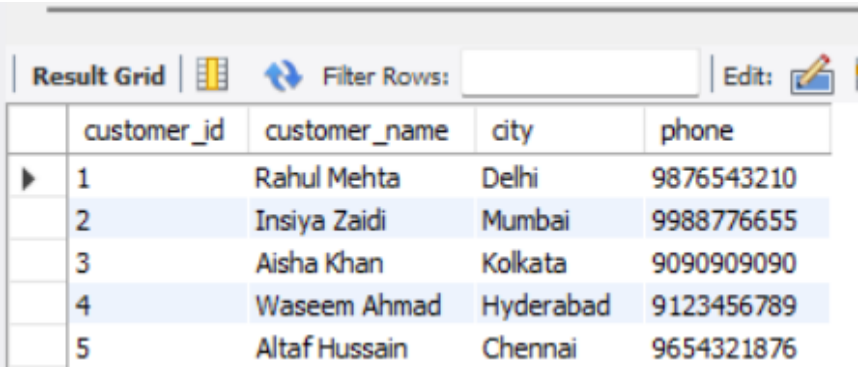
2)

```
CREATE DATABASE shop_inventory;

USE shop_inventory;

CREATE TABLE customers (

   customer_id INT PRIMARY KEY,customer_name VARCHAR(100) NOT NULL,city VARCHAR(50),phone VARCHAR(15)

)

INSERT INTO customers (customer_id, customer_name, city, phone) VALUES

(1, 'Rahul Mehta', 'Delhi', '9876543210'),(2, 'Insiya Zaidi', 'Mumbai', '9988776655'),(3, 'Aisha Khan', 'Kolkata', '9090909090'),(4, 'Waseem Ahmad', 'Hyderabad', '9123456789'),(5, 'Altaf Hussain', 'Chennai', '9654321876');


CREATE TABLE products (

   product_id INT PRIMARY KEY,  product_name VARCHAR(100),price DECIMAL(10,2),stock INT

);

INSERT INTO products (product_id, product_name, price, stock) VALUES

(101, 'Laptop', 55000.00, 15),(102, 'Smartphone', 22000.00, 30),(103, 'Headphones', 1800.00, 50),(104, 'Keyboard', 750.00, 40),(105, 'Mouse', 450.00, 60);
```

DELETE FROM customers WHERE customer_id IS NULL OR customer_name IS NULL;

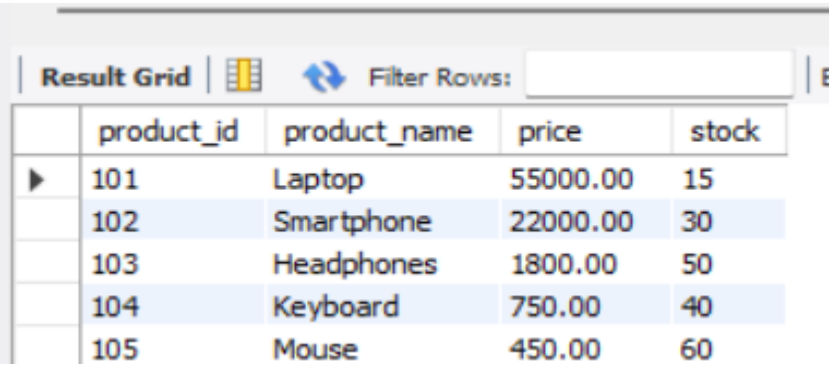
SELECT * FROM customers;

SELECT * FROM products;



Result Grid | Filter Rows: | Edit:

customer_id	customer_name	city	phone
1	Rahul Mehta	Delhi	9876543210
2	Insiya Zaidi	Mumbai	9988776655
3	Aisha Khan	Kolkata	9090909090
4	Waseem Ahmad	Hyderabad	9123456789
5	Altaf Hussain	Chennai	9654321876



Result Grid | Filter Rows:

product_id	product_name	price	stock
101	Laptop	55000.00	15
102	Smartphone	22000.00	30
103	Headphones	1800.00	50
104	Keyboard	750.00	40
105	Mouse	450.00	60

PRACTICAL NO -3

TOPIC –FOREIGN KEY CONSTRAINT

1)

```
CREATE DATABASE college_system;
USE college_system;
CREATE TABLE departments (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(100) NOT NULL
);
INSERT INTO departments (dept_name) VALUES
('Computer Engineering'),
('Electronics'),
('Mechanical'),
('Civil');
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    student_name VARCHAR(100) NOT NULL,
```

```
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);
INSERT INTO students (student_name, dept_id) VALUES
('Insiya', 1),('Faiz', 2),('Arshad', 1),('Kahkashan', 3),('Aitraf', 2),('Madiha', 4);
SELECT * FROM departments;SELECT * FROM students;
```

dept_id	dept_name
1	Computer Engineering
2	Electronics
3	Mechanical
4	Civil

Result Grid | Filter Rows:

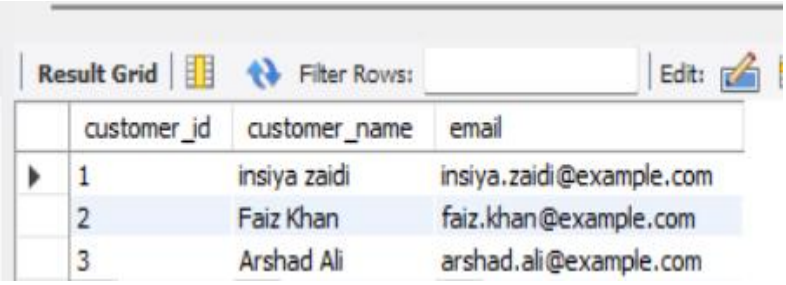
student_id	student_name	dept_id
1	Insiya	1
2	Faiz	2
3	Arshad	1
4	Kahkashan	3
5	Aitraf	2
6	Madiha	4

2)

```
CREATE DATABASE shop_db;
USE shop_db;
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(100) NOT NULL,
```

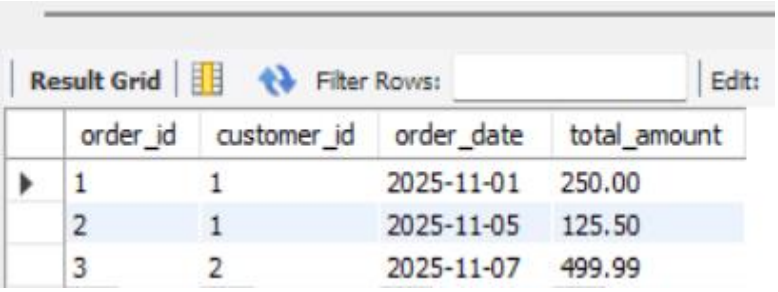
```
    email VARCHAR(120) UNIQUE
);
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2) DEFAULT 0.00,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
        ON DELETE CASCADE
        ON UPDATE RESTRICT
);
INSERT INTO customers (customer_name, email) VALUES
('insiya zaidi', 'insiya.zaidi@example.com'),
('Faiz Khan', 'faiz.khan@example.com'),
('Arshad Ali', 'arshad.ali@example.com');
INSERT INTO orders (customer_id, order_date, total_amount) VALUES
(1, '2025-11-01', 250.00),
(1, '2025-11-05', 125.50),
(2, '2025-11-07', 499.99);
SELECT * FROM customers;
SELECT * FROM orders;
```

```
SELECT o.order_id, o.order_date, o.total_amount, c.customer_name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```



Result Grid | Filter Rows: | Edit:

customer_id	customer_name	email
1	insiya zaidi	insiya.zaidi@example.com
2	Faiz Khan	faiz.khan@example.com
3	Arshad Ali	arshad.ali@example.com



Result Grid | Filter Rows: | Edit:

order_id	customer_id	order_date	total_amount
1	1	2025-11-01	250.00
2	1	2025-11-05	125.50
3	2	2025-11-07	499.99

PRACTICAL NO -4

TOPIC –ALTER TABLE,DROP TABLE , UPDATE TABLE

1) Write SQL queries to create a users database, insert records, alter the table by adding a new column, update a user's city, and then drop the table.

```
CREATE DATABASE demo_db;
USE demo_db;
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    student_name VARCHAR(100),
    city VARCHAR(50)
);
INSERT INTO students (student_name, city) VALUES
('insiya zaidi', 'Delhi'),
('arshad zaidi', 'Mumbai'),
```

('Faiz Khan', 'Lucknow');

select * from students ;

ALTER TABLE students

ADD email VARCHAR(120);ALTER TABLE students MODIFY city VARCHAR(100);ALTER TABLE students

CHANGE student_name full_name VARCHAR(100);

ALTER TABLE students RENAME TO student_info;SET SQL_SAFE_UPDATES = 0; UPDATE student_info SET email = 'insiya.zaidi@example.com'

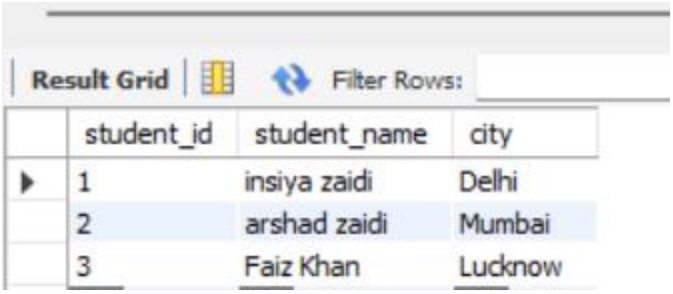
WHERE full_name = 'insiya zaidi';UPDATE student_info SET email = 'arshad.zaidi@example.com'WHERE full_name = 'arshad zaidi';UPDATE student_info

SET email = 'khan@example.com'

WHERE full_name = 'Faiz khan';UPDATE student_info

SET city = 'Hyderabad'WHERE full_name = 'arshad zaidi';DROP TABLE student_info;SELECT * FROM student_info;

BEFORE ALTER --



Result Grid | Filter Rows:

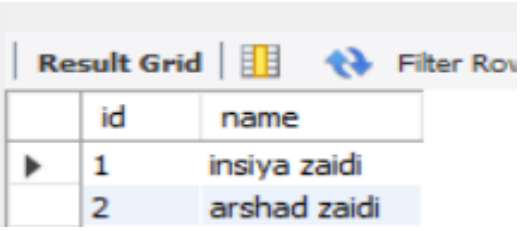
student_id	student_name	city
1	insiya zaidi	Delhi
2	arshad zaidi	Mumbai
3	Faiz Khan	Lucknow

2)**Write SQL commands to create a database named test_db, create a users table, insert records, display the table, alter the table by adding a city column, update the city for a specific user, display only that updated user, and finally drop the users table**

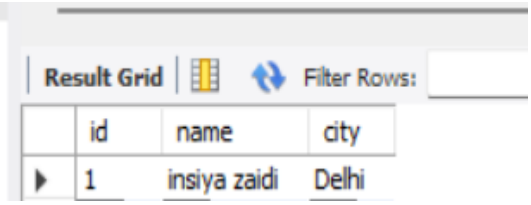
```
CREATE DATABASE test_db;

USE test_db;

CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100)
);

INSERT INTO users (name) VALUES
('insiya zaidi'),
('arshad zaidi');select * from users;ALTER TABLE users ADD city VARCHAR(50);

UPDATE users
```

SET city = 'Delhi' WHERE name = 'insiya zaidi'; select * from users where name = "insiya zaidi";DROP TABLE users;

BEFORE -



AFTER -



PRACTICAL NO -5

TOPIC –AGGREGATION FUNCTION

1) **Write a query to find the count of students and the average marks for every subject, grouped by the subject name.**

```
CREATE DATABASE agg_db3;
USE agg_db3;
CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    subject VARCHAR(50),
    marks INT
);
INSERT INTO students (name, subject, marks) VALUES
('insiya zaidi', 'Math', 90),
```

('Faiz Khan', 'Math', 85),

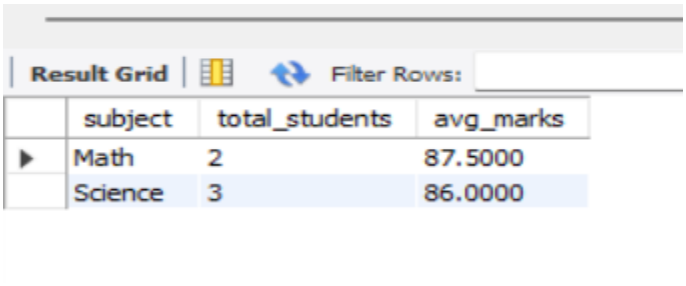
('Arshad Zaidi', 'Science', 78),

('Madiha', 'Science', 88),

('insiya zaidi', 'Science', 92);

SELECT

subject, COUNT(*) AS total_students, AVG(marks) AS avg_marks FROM students GROUP BY subject;



subject	total_students	avg_marks
Math	2	87.5000
Science	3	86.0000

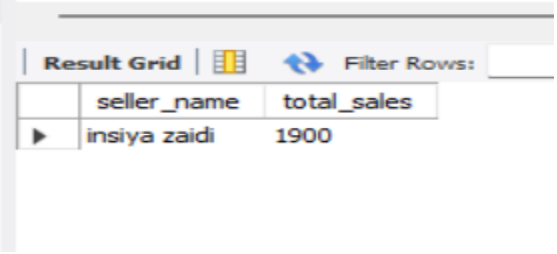
2) Write a query to calculate the total sale amount per seller and filter the results to show only sellers having total sales above 1000

CREATE DATABASE agg_db4;

USE agg_db4;

CREATE TABLE sales (

```
    sale_id INT PRIMARY KEY
AUTO_INCREMENT,seller_name
VARCHAR(100),sale_amount INT

);

INSERT INTO sales (seller_name, sale_amount) VALUES

('insiya zaidi', 1200),('Faiz Khan', 800),('Arshad Zaidi',
500),('insiya zaidi', 700),('Madiha', 900);

SELECT

    seller_name, SUM(sale_amount) AS total_sales

FROM sales  GROUP BY seller_name HAVING
SUM(sale_amount) > 1000;
```



PRACTICAL NO -6

TOPIC –JOIN

```
CREATE DATABASE sailing_db;
USE sailing_db;
CREATE TABLE sailors (
  sid INT PRIMARY KEY,  sname VARCHAR(100),rating INT, age INT
);
CREATE TABLE boats (
   bid INT PRIMARY KEY, bname VARCHAR(100), color VARCHAR(50)
);
CREATE TABLE reserves (
   sid INT, bid INT, day DATE,
   PRIMARY KEY (sid, bid, day),
   FOREIGN KEY (sid) REFERENCES sailors(sid),
   FOREIGN KEY (bid) REFERENCES boats(bid)
);
INSERT INTO sailors (sid, sname, rating, age) VALUES
(1, 'John', 7, 25),
(2, 'Bob', 5, 30),
```

(3, 'Mary', 9, 22),(4, 'Susan', 3, 28),(5, 'Tom', 5, 20),(6, 'Kelly', 7, 24);

INSERT INTO boats (bid, bname, color) VALUES

(10, 'Sea Breeze', 'red'),(11, 'Black Pearl', 'black'),(12, 'Red Dragon', 'red'),(13, 'Wave Rider', 'blue');

INSERT INTO reserves (sid, bid, day) VALUES

(2, 10, '2025-11-01'), (1, 11, '2025-11-03'),(3, 12, '2025-11-05'),

(5, 10, '2025-11-07'), (4, 13, '2025-11-09');

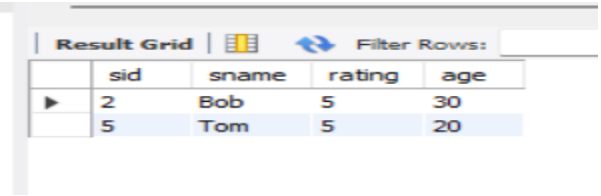
Q1: Find all information of sailors who have reserved boat no 10.

SELECT DISTINCT s.*

FROM sailors s

JOIN reserves r ON s.sid = r.sid

JOIN boats b ON r.bid = b.bid WHERE b.bid = 10;


Result Grid | Filter Rows:

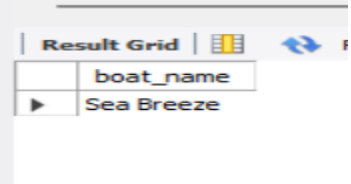
sid	sname	rating	age
2	Bob	5	30
5	Tom	5	20

Q2: Find the name of boat reserved by Bob.

SELECT DISTINCT b.bname AS boat_name

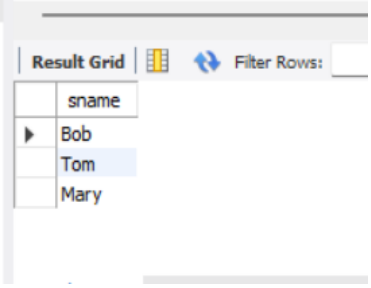
FROM sailors s

```
JOIN reserves r ON s.sid = r.sid
JOIN boats b ON r.bid = b.bid
WHERE s.sname = 'Bob';
```



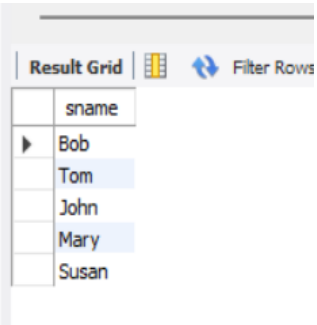
Q3: Find the names of sailors who have reserved a red boat.

```
SELECT DISTINCT s.sname
FROM sailors s
JOIN reserves r ON s.sid = r.sid
JOIN boats b ON r.bid = b.bid
WHERE b.color = 'red';
```



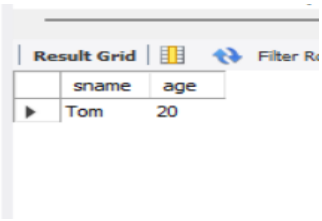
Q4: Find the names of sailors who have reserved at least one boat.

```
SELECT DISTINCT s.sname

FROM sailors s

JOIN reserves r ON s.sid = r.sid;
```



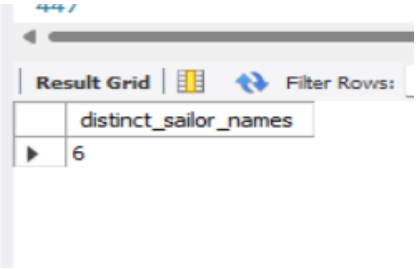
Q5: Find the name and age of the youngest sailor.

```
SELECT sname, age

FROM sailors

ORDER BY age ASC
```



Q6: Count the number of different sailor names.

```
SELECT COUNT(DISTINCT sname) AS distinct_sailor_names

FROM sailors;
```



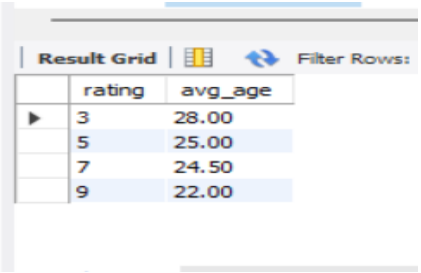
Q7: Find the average age of sailors for each rating level.

SELECT rating, ROUND(AVG(age),2) AS avg_age

FROM sailors

GROUP BY rating

ORDER BY rating;



PRACTICAL NO -7

TOPIC –SUBQUERY

```
DROP DATABASE IF EXISTS subquery_db;

CREATE DATABASE subquery_db;

USE subquery_db;

CREATE TABLE departments (dept_id INT PRIMARY KEY
AUTO_INCREMENT dept_name VARCHAR(100)

);

CREATE TABLE employees (

  emp_id INT PRIMARY KEY AUTO_INCREMENT,

  emp_name VARCHAR(100),dept_id INT,  salary INT,  age INT,

  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)

);

CREATE TABLE projects (

  proj_id INT PRIMARY KEY AUTO_INCREMENT,

  proj_name VARCHAR(100) budget INT

);


CREATE TABLE assignments (
```

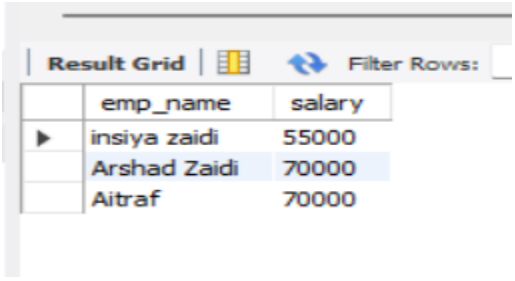
```
 assign_id INT PRIMARY KEY AUTO_INCREMENT,  emp_id INT,
 proj_id INT,hours INT, FOREIGN KEY (emp_id) REFERENCES
employees(emp_id)  FOREIGN KEY (proj_id) REFERENCES
projects(proj_id)
);
INSERT INTO departments (dept_name) VALUES
('HR'),('Engineering'),('Sales'),('Marketing');

INSERT INTO employees (emp_name, dept_id, salary, age) VALUES
('insiya zaidi', 2, 55000, 24),('Arshad Zaidi', 2, 70000, 29),('Faiz Khan', 3,
48000, 26),('Madiha', 3, 52000, 23),('Kahkashan', 4, 45000, 27),('Aitraf',
2, 70000, 31),('Ravi', 1, 40000, 35);

INSERT INTO projects (proj_name, budget) VALUES
('Alpha', 120000),('Beta', 45000),('Gamma', 90000),('Delta', 200000);

INSERT INTO assignments (emp_id, proj_id, hours) VALUES
(1, 1, 40),(2, 1, 60),(2, 3, 30),(3, 2, 20),(4, 3, 50),(6, 4, 120),(5, 4, 10);
```

Q1 .Find the employees whose salary is greater than the average salary of all employees.

SELECT emp_name, salary

FROM employees

WHERE salary > (SELECT AVG(salary) FROM employees);

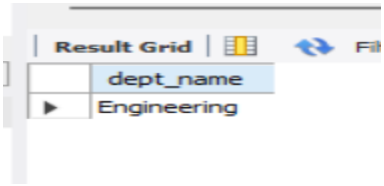


Result Grid | Filter Rows:

emp_name	salary
insiya zaidi	55000
Arshad Zaidi	70000
Aitraf	70000

Q2.Find the departments that have more than 2 employees.

SELECT dept_name FROM department WHERE dept_id IN (SELECT dept_id FROM employees GROUP BY dept_id HAVING COUNT(*) > 2);

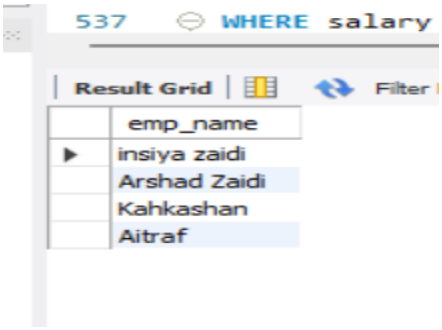


Result Grid | Fil

dept_name
Engineering

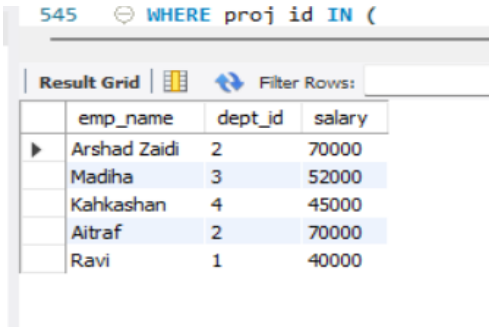
Q3.Find the names of employees who are working on projects whose budget is greater than 100000.

SELECT DISTINCT emp_name FROM employees WHERE emp_id IN (SELECT emp_id FROM assignment WHERE proj_id IN (SELECT proj_id FROM projects WHERE budget > 100000));



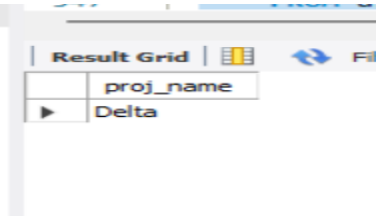
Q4.Find the employees who have the maximum salary in their department.

```
SELECT emp_name, dept_id, salary FROM employees WHERE salary = (
   SELECT MAX(salary) FROM employees WHERE dept_id = e.dept_id
);
```



Q5.Find the projects whose total assigned hours exceed 100.

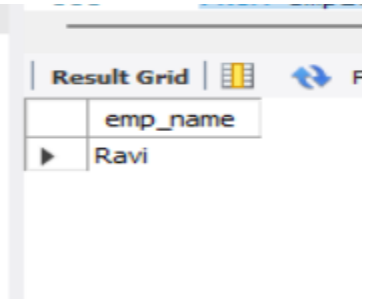
```
SELECT proj_name FROM projects WHERE proj_id IN (
   SELECT proj_id  FROM assignment GROUP BY proj_id   HAVING SUM(hours) > 100
);
```



Q6.Find the employees who are not assigned to any project.

SELECT emp_name FROM employees

WHERE emp_id NOT IN (SELECT emp_id FROM assignments);



PRACTICAL NO -8

TOPIC –NESTED QUERY

DROP DATABASE IF EXISTS

```
CREATE DATABASE nested_db2;

USE nested_db2;

CREATE TABLE customers (

  cust_id INT PRIMARY KEY AUTO_INCREMENT,cust_name
VARCHAR(100),city VARCHAR(50)

);

CREATE TABLE categories (

  cat_id INT PRIMARY KEY AUTO_INCREMENT, cat_name VARCHAR(100)

);

CREATE TABLE products (

  prod_id INT PRIMARY KEY AUTO_INCREMENT,prod_name
VARCHAR(100),

  cat_id INT,
```

```
  price DECIMAL(10,2), FOREIGN KEY (cat_id) REFERENCES
categories(cat_id)

);


CREATE TABLE orders (

  ord_id INT PRIMARY KEY AUTO_INCREMENT,cust_id INT,ord_date
DATE,FOREIGN KEY (cust_id) REFERENCES customers(cust_id)

);

CREATE TABLE order_items (

  item_id INT PRIMARY KEY AUTO_INCREMENT, ord_id INT,prod_id INT,
qty INT,FOREIGN KEY (ord_id) REFERENCES orders(ord_id),FOREIGN KEY
(prod_id) REFERENCES products(prod_id)

);

INSERT INTO customers (cust_name, city) VALUES

('insiya zaidi','Delhi'),('Arshad Zaidi','Mumbai'),('Faiz
Khan','Lucknow'),('Madiha','Noida'),('Ravi','Delhi');


INSERT INTO categories (cat_name) VALUES

('Electronics'),('Books'),('Home');

INSERT INTO products (prod_name, cat_id, price) VALUES

('Smartphone', 1, 25000.00),('Laptop', 1, 55000.00),('Novel A', 2,
499.00),('Cookbook', 2, 799.00),('Mixer', 3, 4500.00),('Vacuum', 3,
8200.00);
```

INSERT INTO orders (cust_id, ord_date) VALUES

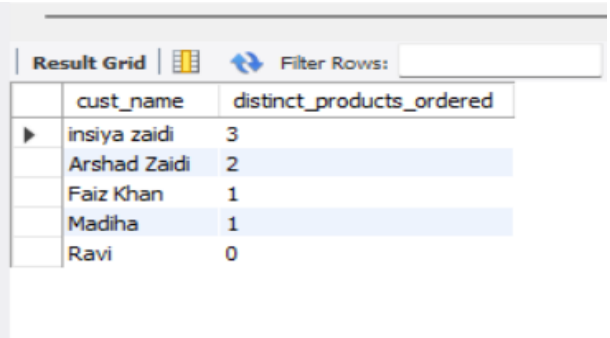
(1,'2025-10-01'),(1,'2025-10-15'),(2,'2025-09-20'),(3,'2025-11-01'),(4,'2025-11-03');

INSERT INTO order_items (ord_id, prod_id, qty) VALUES

(1,1,1), (1,3,2), (2,2,1), (3,4,1), (3,6,1), (4,5,1), (5,3,1);

Q1 — For each customer, show their name and the number of distinct products they ordered

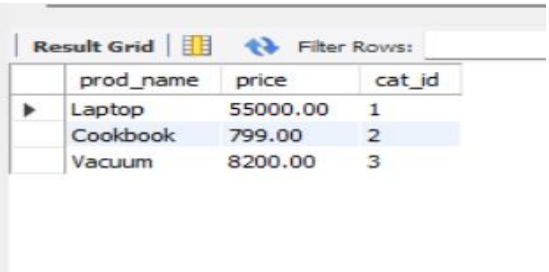
```
SELECT c.cust_name,
(SELECT COUNT(DISTINCT oi.prod_id)
FROM orders o
JOIN order_items oi ON o.ord_id = oi.ord_id
WHERE o.cust_id = c.cust_id) AS distinct_products_ordered
FROM customers c;
```



cust_name	distinct_products_ordered
insiya zaidi	3
Arshad Zaidi	2
Faiz Khan	1
Madiha	1
Ravi	0

Q2 — List products that have price greater than the average price of products in the same category (use a correlated nested query).

```
SELECT p.prod_name, p.price, p.cat_id
FROM products p
WHERE p.price > (
   SELECT AVG(p2.price)
   FROM products P2 WHERE p2.cat_id = p.cat_id
);
```



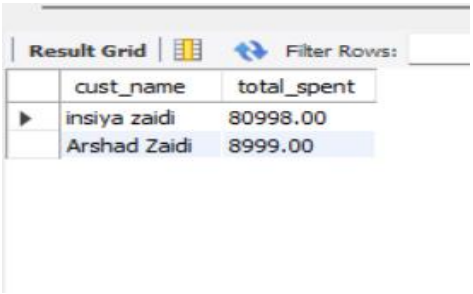
Result Grid | Filter Rows:

prod_name	price	cat_id
Laptop	55000.00	1
Cookbook	799.00	2
Vacuum	8200.00	3

Q3 — Find top 2 customers by total spending using a subquery in FROM (derived table).

```
SELECT t.cust_name, t.total_spent
FROM (
   SELECT c.cust_name,
        SUM(p.price * oi.qty) AS total_spent
   FROM customers c
```

```
    JOIN orders o ON c.cust_id = o.cust_id
    JOIN order_items oi ON o.ord_id = oi.ord_id
    JOIN products p ON oi.prod_id = p.prod_id
    GROUP BY c.cust_id
) AS t ORDER BY t.total_spent DESC LIMIT 2;
```



Result Grid | Filter Rows:

cust_name	total_spent
insiya zaidi	80998.00
Arshad Zaidi	8999.00

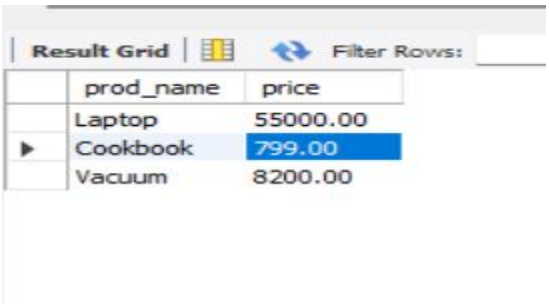
Q4— Find customers who ordered the product named 'Laptop' (use EXISTS subquery).

```
SELECT c.cust_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    JOIN order_items oi ON o.ord_id = oi.ord_id
    JOIN products p ON oi.prod_id = p.prod_id
```

```
    WHERE o.cust_id = c.cust_id AND p.prod_name = 'Laptop'
);
```

Q5) Find the products whose price is higher than the average price of all products in the same category.

```
SELECT prod_name, price
FROM products p
WHERE price > (
  SELECT AVG(price) FROM products WHERE cat_id = p.cat_id
);
```



Result Grid | Filter Rows:

prod_name	price
Laptop	55000.00
Cookbook	799.00
Vacuum	8200.00